

Methodology and Results Report

1) Introduction

This report presents the methodology, design rationale, and evaluation of the CNN–LSTM hybrid forecasting model submitted for the cryptocurrency forecasting competition.

The competition’s task was to predict the next 10 future closing prices for each window of 60 past observations. The data was provided as a large multi-series dataset containing closing prices and trading volumes for several digital assets. The challenge was not only to achieve high numerical accuracy but also to develop a method that generalized across assets with very different scales, volatility regimes, and liquidity levels.

Because of these properties, the model’s design centered on three principles: learning relative rather than absolute patterns, preventing cross-window data leakage, and balancing short-term reactivity with long-term temporal awareness.

2) Data Preparation and Feature Engineering

Each series was divided into fixed windows of 70 timesteps. The first 60 points were treated as the observation period and the next 10 as the prediction horizon. Within each 60-step input window, three channels were constructed: scaled close, scaled log-volume, and local returns.

The closing price was normalized within its own window to remove scale effects that differ across tokens or long time spans. The transformation followed the min–max formulation

$$c_scaled(t) = 2 * ((c_t - c_min) / (c_max - c_min + eps)) - 1$$

$$eps = 1e-8$$

This per-window scaling made the network focus on relative movements, such as trends, oscillations, and reversals, rather than the absolute value of a token’s price. It also ensured that each window became statistically self-contained, eliminating the risk of leakage from global statistics.

Trading volume was first log-transformed before scaling. In raw form, crypto volumes are extremely heavy-tailed: a single liquidity surge can be hundreds of times larger than the median. Applying log1p compresses these extreme spikes while preserving their ordering, so the model can still recognize when market participation rises or falls without letting rare bursts dominate the feature space. The normalized log-volume followed

$$v_log(t) = \log1p(\max(v_t, 0))$$

$$v_scaled(t) = 2 * ((v_log(t) - v_log_min) / (v_log_max - v_log_min + eps)) - 1$$

Finally, local percentage returns were computed inside each window:

$$r_t = ((c_t / c_{t-1}) - 1) * 100$$

$$r_0 = 0$$

Normalizing the closing prices meant that the model no longer had access to the actual scale of price changes : for example, a move from 100 to 110 becomes identical in shape to a move from

300 to 310, even though the first corresponds to a 10% return and the second only about 3.3%. To ensure that the model still accounted for the true return dynamics, the raw percentage returns were explicitly included as both an input feature and the prediction target. This allowed the network to learn from normalized patterns while still being sensitive to the magnitude of returns that those patterns implied.

The model therefore saw, for every window, a tensor of shape (60, 3) representing normalized close, normalized log-volume, and local returns. This feature design was deliberately structured so that the model learned to analyze each window in isolation rather than depend on information from the full series. This reflects the competition's out-of-sample evaluation setup, where new test windows come from unseen time segments or assets, requiring the model to generalize purely from patterns within each window instead of memorizing global trends.

3) Model Architecture and Rationale

The architecture was implemented as a compact CNNLSTM module.

The first stage consisted of two one-dimensional convolutional layers, each with 128 filters and a kernel size of 3 (chosen arbitrarily). These layers scanned across the time axis to detect short-range patterns such as sudden bursts, local reversals, or periodic fluctuations in the input features. ReLU activations introduced non-linearity, and a dropout rate of 0.15 reduced overfitting by randomly omitting neurons during training. The dropout rate was tuned down from 0.4, which caused underfitting and overly smooth forecasts.

Convolutional layers are efficient at handling local correlations; their inclusion allowed the model to recognize repeating motifs without relying on the longer memory of the LSTM.

The output of the convolutional block was passed through two stacked LSTM layers with hidden size 200 (chosen arbitrarily). The LSTM component captured sequential dependencies that span more than a few timesteps, modeling momentum continuation, mean reversion, and volatility clustering. Stacking two LSTMs provided sufficient representational depth to model the interplay between local feature sequences and slower regime changes. The same dropout level (0.15) was applied here as well, which proved sufficient to regularize without dampening responsiveness.

After processing the sequence, the final hidden state of the second LSTM was mapped through a fully connected layer that produced ten outputs, each corresponding to a **predicted return** for one of the next ten timesteps. During inference, these predicted returns were converted back into prices using the relationship

$$p_{\text{hat}}_{\{t+h\}} = p_t * (1 + r_{\text{hat}}_{\{t+h\}} / 100)$$

This approach of predicting returns rather than absolute prices was chosen because returns are inherently scale-invariant and reflect relative movement, making the learning target more consistent across assets and time periods. Predicting raw prices would have forced the model to handle vastly different magnitudes and trends, increasing non-stationarity and making it harder to generalize. By forecasting returns within each window, the model focused purely on directional and proportional changes, the true drivers of predictive performance in volatile crypto markets, while the final price reconstruction preserved interpretability for evaluation.

4) Training Configuration

Training used a straightforward but stable configuration: batch size 32, learning rate 0.001, and 20 epochs. The learning rate was chosen after experimenting with different values. Training beyond 20 epochs showed no meaningful reduction in validation loss, indicating convergence. Dropout at 0.15 acted as the main regularizer, and no additional complex techniques such as learning-rate schedulers or gradient clipping were needed. The entire model was trained on a single GPU (GeForce MX450) using the Adam optimizer and took approximately 1.2hrs.

5) Inference Workflow

At inference time, the model followed exactly the same feature normalization and ordering used during training. Each test group was sorted by `window_id` and `time_step`, then transformed into the (1, 60, 3) tensor format before passing through the trained CNN-LSTM. The predicted returns were immediately converted into price forecasts and aggregated into a dataframe saved as `submission.pkl`.

7) Discussion

Architecturally, the CNN-LSTM combination emerged as the most balanced trade-off between local precision and temporal smoothness. A pure CNN captured fine-grained movements but lost contextual continuity, while a pure LSTM generalized too slowly and smoothed out high-frequency information. The hybrid setup kept the model compact but expressive enough to capture the essential patterns present in the competition dataset.
